

Busca em largura e ordenação topológica

Algoritmos e Estruturas de Dados 2

Prof. Dr. André Yoshiaki Kashiwabara

Departamento de Computação — Universidade Federal de Sergipe

22 de setembro de 2026

Da profundidade à largura

A DFS já nos deu

- alcançabilidade a partir de uma raiz;
- a floresta da busca guardada em `pai []`;
- pré-ordem e pós-ordem, detecção de ciclo;
- custo $\Theta(V + A)$ com listas de adjacência.

Mas ela não responde

- qual é o *menor* número de arcos de s até w ?
- quais vértices estão a exatamente 2 arcos de s ?

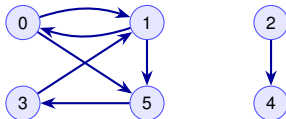
O caminho que a DFS devolve é *um* caminho — em geral não o mais curto. Trocar a pilha por uma **fila** muda isso.

Para aprofundar: DFS (aula anterior) em [Feofiloff, 2020], cap. *Busca em profundidade*, e [Cormen et al., 2009], §22.3; BFS em [Feofiloff, 2020], cap. *Busca em largura* e [Cormen et al., 2009], §22.2.

O grafo desta aula

O mesmo das aulas anteriores: vértices 0..5 e o conjunto de arcos

0-1 0-5 1-0 1-5 2-4 3-1 5-3



Convenção da aula: os vizinhos de um vértice são sempre examinados em ordem crescente — as simulações ficam reproduzíveis.

Para aprofundar: grafo-exemplo e convenções de código de [Feofiloff, 2020], cap. *Estruturas de dados para grafos*.

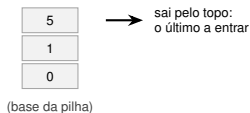
Busca em largura, em uma frase

Visite **todos** os vizinhos do vértice atual antes de avançar para os vizinhos deles: primeiro o que está a 1 arco de s , depois o que está a 2 arcos, e assim por diante.

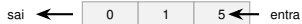
- “Largura” é a prioridade: explorar em **camadas** concêntricas em torno da raiz, como uma onda na água.
- O vértice é marcado *ao entrar na fila*, não ao sair — assim nenhum vértice entra duas vezes.
- A estrutura que produz essa ordem é a **fila** (FIFO): quem foi descoberto primeiro é expandido primeiro.

Para aprofundar: a ideia da busca em largura em [Feofiloff, 2020], cap. *Busca em largura*; [Sedgewick and Wayne, 2011], §4.1; [Levitin, 2012], §3.5.

DFS: pilha (LIFO)



BFS: fila (FIFO)



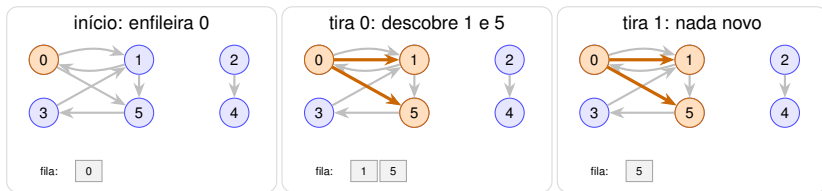
O esqueleto do algoritmo é **idêntico**: retire um vértice da estrutura, examine seus vizinhos, insira os ainda não marcados.

Só a disciplina da estrutura muda — e com ela a ordem de visita, a forma da árvore de busca e as perguntas que o percurso sabe responder.

Na DFS a pilha pode ser implícita (recursão); na BFS a fila é *sempre* explícita.

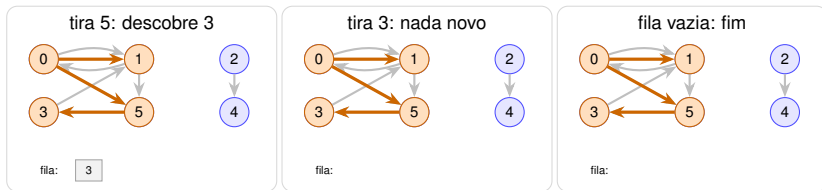
Para aprofundar: a dualidade pilha/fila em [Sedgewick and Wayne, 2011], §4.1; [Feofiloff, 2020], cap. *Busca em largura*; [Levitin, 2012], §3.5.

Simulando a busca em largura



Ao tirar 0 da fila, **ambos** os vizinhos são descobertos e enfileirados de uma vez. Quando 1 sai, seus vizinhos 0 e 5 já estão marcados: nada entra na fila. Laranja = marcado; laranja grosso = arco da árvore.

Para aprofundar: simulações equivalentes em [Feofiloff, 2020], cap. *Busca em largura* e [Sedgewick, 2002], cap. 18.



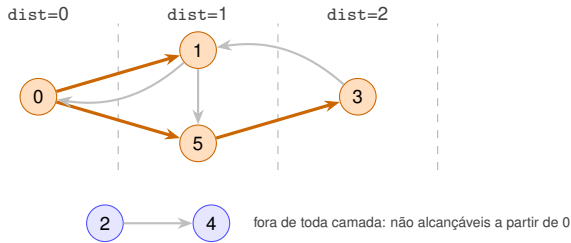
Resultado: alcançáveis a partir de 0 são $\{0, 1, 5, 3\}$ — o mesmo conjunto que a DFS encontrou, e na mesma ordem de visita 0, 1, 5, 3. A *árvore*, porém, é outra: aqui 5 é filho de 0, não de 1.

Para aprofundar: [Feofiloff, 2020], cap. *Busca em largura*; *árvore de busca em largura* em [Cormen et al., 2009], §22.2.

vértice v	0	1	2	3	4	5
$\text{dist}[v]$	0	1	-1	2	-1	1
$\text{pai}[v]$	-1	0	-1	5	-1	0

- $\text{dist}[v]$ é o **número mínimo de arcos** de 0 até v ; -1 significa “não alcançável”.
- As camadas em torno de 0 são $\{0\}$, $\{1, 5\}$, $\{3\}$.
- Seguindo $\text{pai}[]$ de 3 para trás: $3 \leftarrow 5 \leftarrow 0$, o caminho 0 5 3, com **2 arcos**.
- A DFS, no mesmo grafo, daria 0 1 5 3: um caminho válido, com **3 arcos**. A fila é o que garante o mínimo.

Para aprofundar: $\text{dist}[]$ ($= d[v]$), $\text{pai}[]$ ($= \pi[v]$) e a árvore de busca em largura em [Cormen et al., 2009], §22.2 (lema 22.6: o subgrafo de predecessores é uma árvore de caminhos mínimos; teorema 22.7); [Feofiloff, 2020], cap. *Busca em largura*.



Propriedade das camadas: todo arco do grafo liga vértices da mesma camada ou de camadas *consecutivas* — nunca salta uma camada à frente. Se saltasse, o vértice de destino teria sido descoberto mais cedo. É essa propriedade que faz `dist[]` ser mínima. Os vértices 2 e 4 formam um componente separado: sem caminho a partir de 0, ficam fora de toda camada e mantêm `dist = -1`.

Para aprofundar: propriedade das camadas e corretude em [Cormen et al., 2009], §22.2 (teorema 22.5); [Sedgewick and Wayne, 2011], §4.1.

O algoritmo em C

```
static int dist[maxV], pai[maxV];    /* -1 = ainda nao descoberto */

void GRAPHbfs( Graph G, vertex s) {
    for (vertex v = 0; v < G->V; ++v) dist[v] = pai[v] = -1;
    QUEUEinit( G->V);
    dist[s] = 0;  QUEUEput( s);        /* marca s ao ENTRAR na fila */
    while (!QUEUEempty()) {
        vertex v = QUEUEget();        /* o mais antigo da fila */
        for (link a = G->adj[v]; a != NULL; a = a->next)
            if (dist[a->w] == -1) {    /* w ainda nao descoberto */
                dist[a->w] = dist[v] + 1;  pai[a->w] = v;
                QUEUEput( a->w);
            }
    }
}
```

Para aprofundar: código-fonte e variantes em [Feofiloff, 2020], cap. *Busca em largura*; implementação em C em [Sedgewick, 2002], cap. 18; pseudocódigo BFS em [Cormen et al., 2009], §22.2.

```
/* ERRADO: marca w so quando ele SAI da fila */
while (!QUEUEempty()) {
    vertex v = QUEUEget();
    if (dist[v] != -1) continue;          /* ja visitado: descarta */
    /* ... calcula dist[v] aqui ... */
    for (link a = G->adj[v]; a != NULL; a = a->next)
        QUEUEput( a->w);                 /* insere SEM checar */
}
```

Por que evitar

O algoritmo continua correto, mas um mesmo vértice pode entrar na fila uma vez por arco que chega nele. A fila cresce até $\Theta(A)$ em vez de $\Theta(V)$, e o vetor `dist[]` precisa ser recalculado com cuidado na saída. **Regra:** marque no momento da inserção.

Para aprofundar: a mesma distinção aparece em [Sedgewick and Wayne, 2011], §4.1, na comparação entre BFS e Dijkstra.

Afirmção

Ao final de $\text{GRAPH}_{\text{bfs}}(G, s)$, para todo vértice v , $\text{dist}[v]$ é o menor número de arcos de um caminho de s a v (ou -1 , se não há caminho).

Esboço. Dois fatos sustentam a prova. **(i)** Os valores retirados da fila são não decrescentes, e a fila nunca contém ao mesmo tempo valores que difiram por mais de 1 — quem entra recebe $\text{dist}[v]+1$ com v na frente da fila. Logo a busca expande a camada k inteira antes de qualquer vértice da camada $k+1$. **(ii)** Por indução em k : se todos os vértices a distância k recebem $\text{dist} = k$, então todo vértice a distância $k+1$ é vizinho de algum deles, ainda não estava marcado quando essa camada foi expandida (senão sua distância seria $\leq k$) e portanto recebe $k+1$.

Para aprofundar: prova completa (teorema 22.5 e lema 22.3) em [Cormen et al., 2009], §22.2.

	Matriz de adjacências	Listas de adjacência
Inicializar os vetores	$\Theta(V)$	$\Theta(V)$
Examinar os vizinhos de v	$\Theta(V)$	$\Theta(\text{grau de saída de } v)$
Total	$\Theta(V^2)$	$\Theta(V + A)$

- Mesmo custo da DFS, pela mesma razão: cada vértice entra na fila no máximo uma vez, cada arco é examinado no máximo uma vez.
- Espaço extra $\Theta(V)$: os vetores `dist[]` e `pai[]` mais a fila. A fila é a única diferença de espaço — e ela pode conter até V vértices (uma camada inteira).
- `QUEUEput/QUEUEget` em tempo constante: fila circular sobre um vetor de V posições.

Para aprofundar: análise por agregação em [Cormen et al., 2009], §22.2; esparsidade em [Levitin, 2012], §3.5.

Resolve

- caminho com o *menor número de arcos*;
- vértices a exatamente k arcos da raiz;
- alcançabilidade e componentes (varrendo $0..V - 1$);
- o diâmetro de um grafo pequeno (uma BFS por vértice).

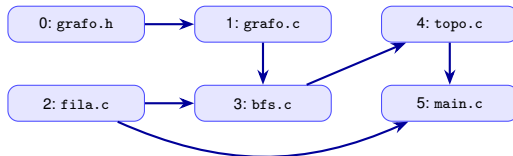
Regra prática: pergunta sobre *distância* pede fila; pergunta sobre *estrutura* (ciclos, ordem, componentes) pede pilha.

Para aprofundar: BFS como caso particular de Dijkstra com pesos unitários em [Cormen et al., 2009], §24.3; [Sedgewick and Wayne, 2011], §4.4.

Não resolve

- caminho de *menor custo* quando os arcos têm **pesos** distintos — a fila trata todo arco como custo 1; isso será o algoritmo de Dijkstra, na Unidade 3;
- nada que dependa de pós-ordem (classificação de arcos, componentes fortes): ali a DFS é insubstituível.

Grafos acíclicos e ordenação topológica



Cada arco $v \rightarrow w$ significa “ v precisa estar pronto antes de w ”. **A pergunta:** em que ordem escrever os seis módulos deste projeto, um de cada vez, sem nunca começar um antes de tudo de que ele depende? `bfs.c` usa a lista de adjacência (`grafo.c`) e a fila (`fila.c`); `topo.c` reaproveita a varredura de `bfs.c`; `main.c` só liga quando os demais existem. Não é distância, não é alcançabilidade — é uma questão de *ordem compatível com os arcos*. O mesmo problema aparece no `make`, em gerenciadores de pacotes, em planilhas (células dependentes) e no escalonamento de tarefas.

Para aprofundar: ordenação topológica e suas aplicações em [Feofiloff, 2020], cap. *Ordenação topológica*; [Cormen et al., 2009], §22.4.

Definições

Um **DAG** (*directed acyclic graph*) é um grafo dirigido sem ciclos. Uma **ordenação topológica** de um grafo dirigido é uma permutação $v_1, v_2, \dots, v_V$ de seus vértices tal que *todo* arco do grafo vai de um vértice para outro **mais à direita** na sequência: se $v_i \rightarrow v_j$ é arco, então $i < j$.

Teorema (existência)

Um grafo dirigido admite ordenação topológica **se e somente se** é um DAG.

Ida: se houvesse um ciclo $u_1 \rightarrow \dots \rightarrow u_k \rightarrow u_1$, algum arco do ciclo teria que apontar para a esquerda na sequência. *Volta*: os dois algoritmos a seguir *constroem* a ordenação de qualquer DAG — a prova é o próprio algoritmo.

Para aprofundar: definição de DAG e de ordenação topológica em [Feofiloff, 2020], cap. *Ordenação topológica*; [Cormen et al., 2009], §22.4 (lema 22.11 e teorema 22.12); [Knuth, 1997], §2.2.3; [Levitin, 2012], §4.2.

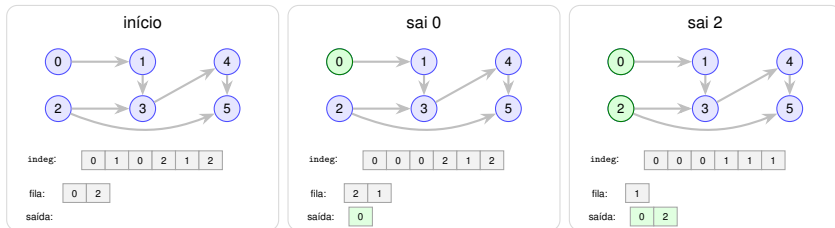
Kahn: graus de entrada e uma fila

O algoritmo em três regras

1. Calcule o **grau de entrada** $\text{indeg}[v]$ de cada vértice: quantos arcos chegam nele.
2. Enfileire todos os vértices com indeg igual a 0 — eles não dependem de ninguém e podem vir primeiro.
3. Retire um vértice v da fila, escreva-o na saída e, para cada vizinho w , decremente $\text{indeg}[w]$; se chegar a 0, enfileire w .

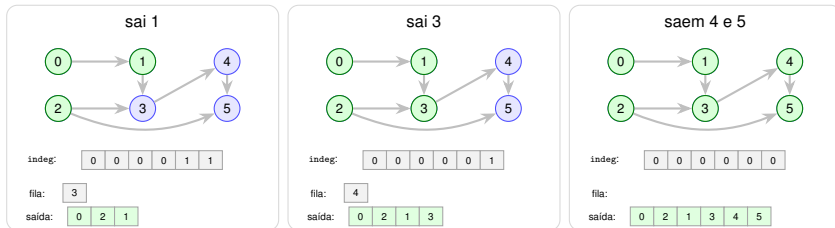
Decrementar $\text{indeg}[w]$ é dizer “mais uma dependência de w ficou pronta”. Quando o contador zera, w está liberado. É a BFS com uma condição de entrada mais forte: o vértice entra na fila quando *todos* os seus predecessores já saíram, não ao ser visto pela primeira vez.

Para aprofundar: algoritmo original em [Kahn, 1962], pp. 558–562; o mesmo método pela remoção de fontes em [Levitin, 2012], §4.2; [Feofiloff, 2020], cap. *Ordenação topológica*.



O vetor `indeg` é mostrado na ordem 0..5. No início apenas 0 (`grafo.h`) e 2 (`fila.c`) têm grau de entrada zero. Ao sair 0, o grau de 1 zera e ele entra na fila *atrás* de 2.

Para aprofundar: [Kahn, 1962], pp. 558–562; a mesma simulação, pelo método de remoção de fontes, em [Levitin, 2012], §4.2.

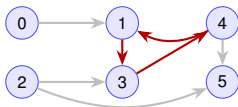


Ordenação obtida: 0, 2, 1, 3, 4, 5 — `grafo.h`, `fila.c`, `grafo.c`, `bfs.c`, `topo.c`, `main.c`. Saíram 6 vértices = V : o grafo é um DAG. Confira nos arcos: nenhum aponta para a esquerda.

Para aprofundar: [Kahn, 1962], pp. 558–562; definição de ordenação topológica em [Cormen et al., 2009], §22.4.

```
/* Escreve em ord[] uma ordenacao topologica; devolve 0 se G tem ciclo. */
int GRAPHtopoKahn( Graph G, vertex ord[]) {
    int indeg[maxV] = {0}, k = 0;
    for (vertex v = 0; v < G->V; ++v)          /* graus de entrada */
        for (link a = G->adj[v]; a != NULL; a = a->next) ++indeg[a->w];
    QUEUEinit( G->V);
    for (vertex v = 0; v < G->V; ++v) if (indeg[v] == 0) QUEUEput( v);
    while (!QUEUEempty()) {
        vertex v = QUEUEget();
        ord[k++] = v;                          /* v entra na saida */
        for (link a = G->adj[v]; a != NULL; a = a->next)
            if (--indeg[a->w] == 0) QUEUEput( a->w);
    }
    return k == G->V;                          /* k < V => ha ciclo */
}
```

Para aprofundar: [Feofiloff, 2020], cap. *Ordenação topológica*; implementação equivalente em [Sedgewick and Wayne, 2011], §4.2; [Ziviani, 2011], §7.3.



Acrescente $4 \rightarrow 1$: `grafo.c` passa a chamar uma função declarada em `topo.c`.

- Graus de entrada: 1 e 3 e 4 nunca zeram — cada um espera por outro do ciclo.
- A fila esvazia com apenas 0 e 2 na saída: $k = 2 < 6 = V$.
- **Os vértices que sobraram** ($\text{indeg}[v] > 0$) são exatamente os que alcançam e são alcançados por um ciclo.

Consequência prática

O mesmo código responde duas perguntas: “qual é uma ordem válida?” e “existe dependência circular?”. É assim que `make` e gerenciadores de pacotes detectam ciclos.

Para aprofundar: detecção de ciclo por contagem da saída em [Kahn, 1962], pp. 558–562; por arco de retorno na DFS em [Cormen et al., 2009], §22.4 (lema 22.11).

A outra via: pós-ordem da DFS

A ideia

Rode a varredura DFS. Quando a chamada $\text{dfsR}(v)$ está *terminando*, todos os vértices alcançáveis a partir de v já foram processados — logo v deve vir **antes** de todos eles. Empilhe os vértices em pós-ordem e leia a pilha de trás para frente.

No DAG dos módulos, a varredura a partir de 0 termina os vértices na ordem

pós-ordem: 5 4 3 1 0 2

e a ordem inversa é

2 0 1 3 4 5

Também é uma ordenação topológica válida — e *diferente* da que Kahn produziu (0, 2, 1, 3, 4, 5). Um DAG pode ter muitas ordenações topológicas.

Para aprofundar: TOPOLOGICAL-SORT por pós-ordem em [Cormen et al., 2009], §22.4 (teorema 22.12); [Feofiloff, 2020], cap. *Ordenação topológica*; [Sedgewick and Wayne, 2011], §4.2; [Levitin, 2012], §4.2.

```
static int cnt2;          /* ultima vaga livre de ord[], do fim ao inicio */
static void dfsRtopo( Graph G, vertex v, vertex ord[]) {
    pre[v] = 0;           /* marca v */
    for (link a = G->adj[v]; a != NULL; a = a->next)
        if (pre[a->w] == -1) dfsRtopo( G, a->w, ord);
    ord[--cnt2] = v;      /* POS-ordem: v ocupa a ultima vaga */
}

void GRAPHtopoDfs( Graph G, vertex ord[]) {
    for (vertex v = 0; v < G->V; ++v) pre[v] = -1;
    cnt2 = G->V;
    for (vertex v = 0; v < G->V; ++v)
        if (pre[v] == -1) dfsRtopo( G, v, ord);
}
```

Escrever em `ord[--cnt2]` é inverter a pós-ordem. O código **supõe** que G é um DAG.

Para aprofundar: TOPOLOGICAL-SORT em [Cormen et al., 2009], §22.4; código em C em [Feofiloff, 2020], cap. *Ordenação topológica*.

	Kahn (fila)	DFS (pós-ordem)
Estrutura auxiliar	fila + vetor indeg	pilha de recursão
Pré-processamento	calcular indeg: $\Theta(V + A)$	nenhum
Custo total	$\Theta(V + A)$	$\Theta(V + A)$
Espaço extra	$\Theta(V)$	$\Theta(V)$ (pilha)
Detecta ciclo	sim, contando a saída	sim, mas exige classificar arcos
Ordem no exemplo	0, 2, 1, 3, 4, 5	2, 0, 1, 3, 4, 5
Boa quando	escalonamento com níveis	a DFS já está no código

Um detalhe útil: Kahn permite escolher *qual* vértice de grau zero sai primeiro. Trocando a fila por uma fila de prioridades obtém-se a ordenação topológica lexicograficamente menor; e o conjunto de vértices que estão na fila ao mesmo tempo é um conjunto de tarefas que poderiam rodar *em paralelo*.

Para aprofundar: comparação dos dois métodos em [Levitin, 2012], §4.2, e [Sedgewick and Wayne, 2011], §4.2; custos em [Cormen et al., 2009], §22.4; [Kahn, 1962], pp. 558–562.

Fechamento

Exercícios

1. Implemente a fila circular sobre um vetor de V posições.
2. Escreva `GRAPHshortestPath( $G$ ,  $s$ ,  $t$ )`: imprime um caminho mínimo de s a t usando `pai[]`, ou avisa que não existe.
3. Adapte `GRAPHbfs()` para a matriz de adjacências e confirme que `dist[]` não muda.
4. Faça `GRAPHtopoKahn()` devolver também os vértices envolvidos em algum ciclo.
5. Num DAG de dependências, calcule o número mínimo de rodadas de compilação paralela.
6. Exiba um DAG com $V = 4$ e exatamente 3 ordenações topológicas.

O tutorial em Jupyter traz esqueletos em C para os itens 1, 2, 4 e 5.

Para aprofundar: exercícios sobre BFS e ordenação topológica em [Feofiloff, 2020], cap. *Busca em largura* e cap. *Ordenação topológica*; [Cormen et al., 2009], §22.2–22.4; [Ziviani, 2011], §7.3.

As cinco ideias centrais

- Trocar a pilha por uma **fila** muda a busca: mesmo código, camadas em vez de mergulho.
- Marque o vértice *ao enfileirar*: `dist[]` sai mínima e a fila nunca passa de V elementos.
- Custo $\Theta(V + A)$ com listas — o mesmo da DFS.
- Uma **ordenação topológica** existe se e somente se o grafo é um DAG; um DAG pode ter várias.
- Kahn e a pós-ordem invertida da DFS resolvem o mesmo problema em $\Theta(V + A)$, e ambos denunciam ciclos.

Próxima aula: revisão da Unidade 1 — corretude, força bruta, geração combinatória e grafos — para a prova de 29/09.

Para aprofundar: BFS em [Feofiloff, 2020], cap. *Busca em largura* e [Cormen et al., 2009], §22.2; ordenação topológica em cap. *Ordenação topológica*, [Kahn, 1962], pp. 558–562 e [Cormen et al., 2009], §22.4.

Conceito	Onde ler
A ideia da BFS e o código em C	[Feofiloff, 2020]; [Sedgewick, 2002], cap. 18
Camadas, <code>dist[]</code> e corretude	[Cormen et al., 2009], §22.2 (lema 22.3, teor. 22.5)
Pilha vs. fila	[Sedgewick and Wayne, 2011], §4.1; [Levitin, 2012], §3.5
Custo $\Theta(V + A)$	[Cormen et al., 2009], §22.2
BFS e pesos: o que falta	[Cormen et al., 2009], §24.3; [Sedgewick and Wayne, 2011], §4.4
DAG e existência da ordenação	[Cormen et al., 2009], §22.4; [Knuth, 1997], §2.2.3
Algoritmo de Kahn	[Kahn, 1962]; [Sedgewick and Wayne, 2011], §4.2
Ordenação por pós-ordem da DFS	[Feofiloff, 2020]; [Cormen et al., 2009], §22.4
Implementações e exercícios	[Ziviani, 2011], §7.3

Leitura mínima da aula: [Feofiloff, 2020], capítulos *Busca em largura* e *Ordenação topológica*.

-  Aho, A. V., Hopcroft, J. E., and Ullman, J. D. (1983).
Data Structures and Algorithms.
Addison-Wesley.
Seção 6.7: o algoritmo das duas buscas para componentes fortemente conexos, atribuído a S. R. Kosaraju (1978, inédito).
-  Bentley, J. (2000).
Programming Pearls.
Addison-Wesley, 2 edition.
-  Blum, M., Floyd, R. W., Pratt, V., Rivest, R. L., and Tarjan, R. E. (1973).
Time bounds for selection.
Journal of Computer and System Sciences, 7(4):448–461.
Artigo original da mediana das medianas: seleção em $O(n)$ no pior caso.
-  Bondy, J. A. and Murty, U. S. R. (2008).
Graph Theory, volume 244 of *Graduate Texts in Mathematics*.
Springer.

-  Cormen, T. H., Leiserson, C. E., Rivest, R. L., and Stein, C. (2009).
Introduction to Algorithms.
MIT Press, 3 edition.
-  Feofiloff, P. (2020).
Algoritmos para grafos (em linguagem C).
Material didático online, DCC/IME-USP.
Capítulos *Grafos* e *Estruturas de dados para grafos*. Acesso em 01 set. 2026.
-  Feofiloff, P., Kohayakawa, Y., and Wakabayashi, Y. (2011).
Uma introdução sucinta à teoria dos grafos.
Texto didático online, IME-USP.
Referência em português para as definições de teoria dos grafos. Acesso em 01 set. 2026.
-  Garey, M. R. and Johnson, D. S. (1979).
Computers and Intractability: A Guide to the Theory of NP-Completeness.
W. H. Freeman.



Gries, D. (1981).
The Science of Programming.
Springer-Verlag.



Held, M. and Karp, R. M. (1962).
A dynamic programming approach to sequencing problems.
Journal of the Society for Industrial and Applied Mathematics, 10(1):196–210.



Hoare, C. A. R. (1961a).
Algorithm 64: Quicksort.
Communications of the ACM, 4(7):321.



Hoare, C. A. R. (1961b).
Algorithm 65: FIND.
Communications of the ACM, 4(7):321–322.
O quickselect, publicado por Hoare no mesmo ano do quicksort.



Hoare, C. A. R. (1969).

An axiomatic basis for computer programming.
Communications of the ACM, 12(10):576–580.



Kahn, A. B. (1962).

Topological sorting of large networks.
Communications of the ACM, 5(11):558–562.
Artigo original do algoritmo de graus de entrada com fila.



Knuth, D. E. (1997).

The Art of Computer Programming, Volume 1: Fundamental Algorithms.
Addison-Wesley, 3 edition.
Seção 2.2.3: ordenação topológica como aplicação de listas ligadas.



Knuth, D. E. (1998).

The Art of Computer Programming, Volume 3: Sorting and Searching.
Addison-Wesley, 2 edition.
Seção 5.3.3: seleção mínima em número de comparações.



Levitin, A. (2012).
Introduction to the Design and Analysis of Algorithms.
Pearson, 3 edition.



Manber, U. (1989).
Introduction to Algorithms: A Creative Approach.
Addison-Wesley.



Musser, D. R. (1997).
Introspective sorting and selection algorithms.
Software: Practice and Experience, 27(8):983–993.
O introselect: híbrido entre quickselect e mediana das medianas.



Sedgewick, R. (2002).
Algorithms in C, Part 5: Graph Algorithms.
Addison-Wesley, 3 edition.
Fonte dos exemplos e exercícios do Capítulo 17 citados por Feofiloff.



Sedgewick, R. and Wayne, K. (2011).

Algorithms.

Addison-Wesley, 4 edition.



Sharir, M. (1981).

A strong-connectivity algorithm and its applications in data flow analysis.

Computers & Mathematics with Applications, 7(1):67–72.

Algoritmo das duas buscas (Kosaraju–Sharir) para componentes fortemente conexos.



Tarjan, R. (1972).

Depth-first search and linear graph algorithms.

SIAM Journal on Computing, 1(2):146–160.

Artigo que estabeleceu a DFS como ferramenta algorítmica de tempo linear.



Ziviani, N. (2010).

Projeto de Algoritmos: com implementações em Pascal e C.

Cengage Learning, 3 edition.



Ziviani, N. (2011).

Projeto de Algoritmos com Implementações em Pascal e C.

Cengage Learning, 3 edition.

Capítulo 7: algoritmos em grafos.